GUIDE 07

Build CRUD APIs for Listings and Reviews and Test with Postman

UniMart Monolithic Full-Stack Lab Series

B.Sc. (Hons) IT - Level 2 | Hands-on Laboratory Practical

Department of Industrial Management
Faculty of Science, University of Kelaniya

Learning outcomes

- Install and organize Postman for local API testing.
- Implement secure CRUD endpoints for Listing and Review features.
- Apply validation, ownership checks, transactions, pagination and consistent errors.
- Create positive and negative automated Postman tests.

Prerequisites

- Guides 03-06 completed.
- A seeded category, users and one completed order for review testing.

Suggested student time: 4-5 hours across two lab sessions

Selected scope: The two CRUD features are Listings and Reviews. Review creation requires a completed order so the implementation remains faithful to the case requirement that reviews follow a transaction. Real card payments are outside this lab; seed a completed order or use a mock payment status.

Part A - Set up Postman

1. Install the current Postman desktop app for the operating system. The desktop app provides the full local-testing experience.
2. Create a workspace named UniMart Lab and a collection named UniMart API v1.
3. Create an environment named UniMart Local with base_url = http://localhost:8080/api/v1 and access_token = empty.
4. Reference variables as {{base_url}} and {{access_token}}.
5. At collection level, set Authorization type Bearer Token and value {{access_token}}. Override auth for public endpoints.
6. Organize folders: Auth, Listings, Reviews, Negative Tests.

Part B - API contract

Method and path	Authentication	Result
POST /auth/register	Public	Create university-email account.
POST /auth/login	Public	Return short-lived access token.
GET /listings	Public	Paged/filterable list.
GET /listings/{id}	Public	Listing details.
POST /listings	Authenticated seller	Create listing owned by current user.
PUT /listings/{id}	Owner/admin	Replace editable listing fields.
DELETE /listings/{id}	Owner/admin	Soft archive listing.
GET /listings/{id}/reviews	Public	Paged reviews for listing/seller.
POST /reviews	Authenticated buyer	Create one review for completed order.
PUT /reviews/{id}	Review author/admin	Update rating/comment.
DELETE /reviews/{id}	Review author/admin	Delete or hide review according to policy.

Part C - Listing model and DTOs

```[Java]
public enum ListingStatus {
  AVAILABLE, RESERVED, SOLD, ARCHIVED
}

@Entity
@Table(name = "listings")
public class Listing {
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @ManyToOne(fetch = FetchType.LAZY, optional = false)
  @JoinColumn(name = "seller_id", nullable = false)
  private User seller;

  @ManyToOne(fetch = FetchType.LAZY, optional = false)
  @JoinColumn(name = "category_id", nullable = false)
  private Category category;

  @Column(nullable = false, length = 160)
  private String title;

  @Lob @Column(nullable = false)
  private String description;

  @Column(nullable = false, precision = 12, scale = 2)
  private BigDecimal price;

  @Enumerated(EnumType.STRING)
  @Column(nullable = false, length = 30)
  private ListingStatus status = ListingStatus.AVAILABLE;

  @Version private Long version;
  private Instant createdAt;
  private Instant updatedAt;
  // Generate constructors/getters/setters in IntelliJ.
}
```

```[Java]
public record ListingRequest(
    @NotBlank @Size(max = 160) String title,
    @NotBlank @Size(max = 5000) String description,
    @NotNull @DecimalMin("0.00") BigDecimal price,
    @NotNull Long categoryId
) {}

public record ListingResponse(
    Long id,
    Long sellerId,
    String sellerName,
    Long categoryId,
    String categoryName,
    String title,
    String description,
    BigDecimal price,
    ListingStatus status,
    Instant createdAt,
    Instant updatedAt
) {}
```

Part D - Listing repository, service and controller

```[Java]
public interface ListingRepository extends JpaRepository<Listing, Long>,
    JpaSpecificationExecutor<Listing> {

  Optional<Listing> findByIdAndStatusNot(Long id, ListingStatus status);
}
```

```java
@Service
public class ListingService {
  private final ListingRepository listings;
  private final CategoryRepository categories;
  private final UserRepository users;
  private final ListingMapper mapper;

  @Transactional
  public ListingResponse create(ListingRequest request, String email) {
    User seller = users.findByUniversityEmail(email)
        .orElseThrow(() -> new ResourceNotFoundException("User not found"));
    Category category = categories.findByIdAndActiveTrue(request.categoryId())
        .orElseThrow(() -> new ResourceNotFoundException("Category not found"));

    Listing listing = new Listing();
    listing.setSeller(seller);
    listing.setCategory(category);
    listing.setTitle(request.title().trim());
    listing.setDescription(request.description().trim());
    listing.setPrice(request.price());
    listing.setStatus(ListingStatus.AVAILABLE);
    listing.setCreatedAt(Instant.now());
    listing.setUpdatedAt(Instant.now());
    return mapper.toResponse(listings.save(listing));
  }

  @Transactional(readOnly = true)
  public ListingResponse get(Long id) {
    Listing listing = listings.findByIdAndStatusNot(id, ListingStatus.ARCHIVED)
        .orElseThrow(() -> new ResourceNotFoundException("Listing not found"));
    return mapper.toResponse(listing);
  }

  @Transactional
  public ListingResponse update(Long id, ListingRequest request, String email) {
    Listing listing = requireOwnedListing(id, email);
    Category category = categories.findByIdAndActiveTrue(request.categoryId())
        .orElseThrow(() -> new ResourceNotFoundException("Category not found"));
    listing.setTitle(request.title().trim());
    listing.setDescription(request.description().trim());
    listing.setPrice(request.price());
    listing.setCategory(category);
    listing.setUpdatedAt(Instant.now());
    return mapper.toResponse(listing);
  }

  @Transactional
  public void archive(Long id, String email) {
    Listing listing = requireOwnedListing(id, email);
    if (listing.getStatus() == ListingStatus.SOLD) {
      throw new ConflictException("Sold listings cannot be deleted");
    }
    listing.setStatus(ListingStatus.ARCHIVED);
  }

  private Listing requireOwnedListing(Long id, String email) {
    Listing listing = listings.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Listing not found"));
    if (!listing.getSeller().getUniversityEmail().equalsIgnoreCase(email)) {
      throw new ForbiddenException("You do not own this listing");
    }
    return listing;
  }
}
```

```[Java]
@RestController
@RequestMapping("/api/v1/listings")
public class ListingController {
  private final ListingService service;

  @GetMapping("/{id}")
  public ListingResponse get(@PathVariable Long id) {
    return service.get(id);
  }

  @PostMapping
  public ResponseEntity<ListingResponse> create(
      @Valid @RequestBody ListingRequest request,
      Authentication authentication) {
    ListingResponse created = service.create(request, authentication.getName());
    return ResponseEntity.status(HttpStatus.CREATED).body(created);
  }

  @PutMapping("/{id}")
  public ListingResponse update(@PathVariable Long id,
      @Valid @RequestBody ListingRequest request,
      Authentication authentication) {
    return service.update(id, request, authentication.getName());
  }

  @DeleteMapping("/{id}")
  @ResponseStatus(HttpStatus.NO_CONTENT)
  public void delete(@PathVariable Long id, Authentication authentication) {
    service.archive(id, authentication.getName());
  }
}
```

Implement GET /listings with Pageable and a Specification for q, categoryId and status. Enforce a maximum page size such as 50 to protect the server.

Part E - Review model and use-case rules

```[Java]
@Entity
@Table(name = "reviews",
       uniqueConstraints = @UniqueConstraint(name = "uk_review_order",
                                             columnNames = "order_id"))
public class Review {
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @OneToOne(fetch = FetchType.LAZY, optional = false)
  @JoinColumn(name = "order_id", nullable = false)
  private Order order;

  @ManyToOne(fetch = FetchType.LAZY, optional = false)
  @JoinColumn(name = "reviewer_id", nullable = false)
  private User reviewer;

  @ManyToOne(fetch = FetchType.LAZY, optional = false)
  @JoinColumn(name = "reviewee_id", nullable = false)
  private User reviewee;

  @Column(nullable = false) private Integer rating;
  @Column(length = 1000) private String comment;
  private Instant createdAt;
  private Instant updatedAt;
}
```

```[Java]
public record ReviewCreateRequest(
    @NotNull Long orderId,
    @NotNull @Min(1) @Max(5) Integer rating,
    @Size(max = 1000) String comment
) {}

public record ReviewUpdateRequest(
    @NotNull @Min(1) @Max(5) Integer rating,
    @Size(max = 1000) String comment
) {}
```

```
[Java]
@Transactional
public ReviewResponse create(ReviewCreateRequest request, String email) {
  Order order = orders.findById(request.orderId())
      .orElseThrow(() -> new ResourceNotFoundException("Order not found"));

  if (order.getStatus() != OrderStatus.COMPLETED) {
    throw new ConflictException("Only completed orders can be reviewed");
  }
  if (!order.getBuyer().getUniversityEmail().equalsIgnoreCase(email)) {
    throw new ForbiddenException("Only the buyer can review this order");
  }
  if (reviews.existsByOrderId(order.getId())) {
    throw new ConflictException("This order already has a review");
  }

  Review review = new Review();
  review.setOrder(order);
  review.setReviewer(order.getBuyer());
  review.setReviewee(order.getListing().getSeller());
  review.setRating(request.rating());
  review.setComment(normalize(request.comment()));
  review.setCreatedAt(Instant.now());
  review.setUpdatedAt(Instant.now());
  return mapper.toResponse(reviews.save(review));
}
```

Part F - Security filter chain

```
[Java]
@Bean
SecurityFilterChain apiSecurity(HttpSecurity http) throws Exception {
  return http
      .csrf(csrf -> csrf.disable())
      .cors(Customizer.withDefaults())
      .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
      .authorizeHttpRequests(auth -> auth
          .requestMatchers(HttpMethod.POST, "/api/v1/auth/**").permitAll()
          .requestMatchers(HttpMethod.GET, "/api/v1/listings/**").permitAll()
          .requestMatchers(HttpMethod.GET, "/api/v1/reviews/**").permitAll()
          .requestMatchers("/actuator/health").permitAll()
          .anyRequest().authenticated())
      .oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()))
      .build();
}
```

Security context: Disabling CSRF is appropriate for stateless bearer-token API requests when the access token is sent in the Authorization header. If authentication is moved to cookies, reassess CSRF protection.

Part G - Postman requests

```
[HTTP]
POST {{base_url}}/auth/login
Content-Type: application/json

{
  "email": "seller@stu.kln.ac.lk",
  "password": "ChangeMe123!"
}
```

```
[Postman test]
// Login request: Scripts > Post-response
pm.test("Login succeeds", () => pm.response.to.have.status(200));
const json = pm.response.json();
pm.expect(json.accessToken).to.be.a("string");
pm.environment.set("access_token", json.accessToken);
```

```
[HTTP]
POST {{base_url}}/listings
Authorization: Bearer {{access_token}}
Content-Type: application/json

{
  "title": "Data Structures Textbook",
  "description": "Second edition, clean condition.",
  "price": 2500.00,
  "categoryId": 1
}
```

```
[Postman test]
pm.test("Created", () => pm.response.to.have.status(201));
const body = pm.response.json();
pm.expect(body.id).to.be.a("number");
pm.expect(body.status).to.eql("AVAILABLE");
pm.environment.set("listing_id", body.id);
```

```
[HTTP]
POST {{base_url}}/reviews
Authorization: Bearer {{access_token}}
Content-Type: application/json

{
  "orderId": {{completed_order_id}},
  "rating": 5,
  "comment": "Item matched the description."
}
```

Part H - Required negative tests

Test	Expected
Create listing without token	401 Unauthorized.
Create listing with blank title	400 with fieldErrors.title.
Update another seller's listing	403 Forbidden.
Get archived/non-existent listing	404 Not Found.
Review an order that is not completed	409 Conflict.
Review another buyer's order	403 Forbidden.
Create a second review for the same order	409 Conflict.
Rating outside 1-5	400 validation error.
Use expired/malformed JWT	401 Unauthorized.

Part I - Collection runner evidence

7. Add all requests to the collection in dependency order: login, create listing, get, update, delete, create review, update, delete, negative tests.
8. Use environment variables populated by post-response scripts for created IDs.
9. Run the collection and require all tests to pass.
10. Export the collection without local secret values and commit it under docs/postman or a dedicated documentation repository.
11. Include a screenshot or exported JSON test result as lab evidence, but redact tokens and passwords.

Completion checklist

☐ Listing CRUD uses DTO validation, service transactions and ownership checks.

☐ Listing deletion is a safe archive operation.

☐ Review CRUD enforces completed-order and one-review-per-order rules.

☐ Controllers do not access repositories directly.

☐ Postman environment stores base URL and local token values.

☐ Positive and negative collection tests pass.